Machine Coding: Tab Component

Problem: Build a reusable, accessible Tab component
Tech stack: React 19 + TypeScript (Vite)
Time limit: 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** what must work and what is out of scope. Write the assumptions down.
2. **List interactions**: every click, key and input, and what it should do. This is your checklist.
3. **Sketch components** and who owns which state.
4. **Design state and data** as small as possible. Derive everything else.
5. **Agree the contract** (props / API shape) before writing JSX.
6. **Find the hard parts** early so they get time, not the last 5 minutes.
7. **Build the simplest working version**, then add one working layer at a time.
8. **Cover states and edge cases**: empty, disabled, invalid input.
9. **Wrap up**: walk through trade-offs and what you would add next.

Time budget (60 minutes)

Phase	Minutes
Requirements, questions, assumptions	5
Interactions, components, state, props contract	8
Basic tabs: click to switch, render active panel	10
Accessibility: roles, ids, roving tabindex	8
Keyboard navigation (arrows, Home/End, skip disabled)	10
Controlled / uncontrolled, fallback, keepMounted	9
Styling, edge cases, manual test	7
Wrap-up and trade-offs	3

Think out loud

Say what you are doing and why before you type: *"I'll keep only the active id in state and derive the index, so they can never go out of sync."* The interviewer grades your reasoning as much as the code. If you get stuck, say what you are trying, so they can nudge you.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Render a row of tab buttons and one content panel.
- Clicking a tab shows its panel and marks it active.
- Exactly one tab is active at a time; the first enabled tab is active by default.
- Tabs can be **disabled**: they cannot be clicked or reached with the keyboard.
- Keyboard: **Left/Right** arrows move between tabs (wrap around), **Home/End** jump to first/last enabled tab.
- The parent can read the active tab (`onChange`) and optionally control it (`activeId`).

Non-functional requirements

Area	What it means here
Performance	Switching is O(n) on a handful of tabs; only the active panel renders by default (lazy).
Scalability	Data-driven `items` array, so 3 or 30 tabs use the same code. Horizontal scroll if they overflow.
Error handling	Invalid or disabled `defaultActiveId` falls back to the first enabled tab; empty list shows a message instead of crashing.
Accessibility	WAI-ARIA tabs pattern: `tablist` / `tab` / `tabpanel` , `aria-selected` , `aria-controls` , roving `tabIndex` , visible focus ring.
Reusability	One `<Tabs>` component with a small props API; works controlled or uncontrolled.
Extensibility	Easy to add vertical orientation, manual activation, closable tabs or URL sync later.

Clarifying questions for the interviewer

1. Should tab data come as props (`items` array) or as composable children (`<Tab>` , `<TabPanel>`)?
2. Should arrow keys **activate** a tab immediately (automatic) or only **focus** it until Enter/Space (manual)?
3. When I switch away, should the old panel keep its state (e.g. typed input), or can it unmount?
4. Do we need disabled tabs? Should the active tab be reflected in the URL?
5. Horizontal only, or vertical tabs too?
6. Any library allowed, or plain React + CSS?

Assumptions we will build with

- Data-driven API: `items: { id, label, content, disabled? }[]` .
- **Automatic activation**: arrow keys move focus *and* select (the ARIA recommendation when panels are cheap to show).
- Panels unmount by default (lazy); `keepMounted` keeps them in the DOM with `hidden` so state survives.
- Horizontal tabs, plain React + CSS, no URL sync.
- Supports both **uncontrolled** (`defaultActiveId`) and **controlled** (`activeId` + `onChange`) use.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	First enabled tab (or `defaultActiveId`) is active, its panel is shown
Click an enabled tab	That tab becomes active, its panel shows, `onChange(id)` fires
Click the already active tab	Nothing changes, `onChange` does not fire
Click a disabled tab	Nothing happens (native `disabled` button)
`Tab` key into the tablist	Focus lands on the **active** tab only (one tab stop)
`Tab` key again	Focus moves into the active panel, not to the next tab
`ArrowRight` / `ArrowLeft`	Next / previous enabled tab is focused and activated; wraps at the ends
Arrow onto a disabled tab	It is skipped
`Home` / `End`	First / last enabled tab is focused and activated
Any other key	Ignored, default browser behaviour kept
Type in a panel, switch, come back (`keepMounted`)	Typed value is still there
`defaultActiveId` is missing or disabled	Falls back to the first enabled tab
`items` is empty	"No tabs to show." message
Controlled: parent changes `activeId`	Component shows that tab

This table is the implementation and test checklist.

3. Design the architecture

Module	Responsibility
`App`	Demo page: owns `activeId` (controlled example), shows the active tab name
`Tabs`	Renders tablist + panels, owns selection logic, keyboard handling, ARIA wiring
`getNextIndex`	Pure function: next enabled index in a direction, with wrap-around
`data/tabs`	Sample tab items

```
App
 └── Tabs (items, activeId, onChange, keepMounted)
      ├── div[role=tablist]  ── onKeyDown → getNextIndex()
      │     └── button[role=tab] × n
      └── div[role=tabpanel]  (active only, or all with `hidden`)
```

Start simple: one `Tabs` component. A compound API (`<Tabs><TabList><Tab/></TabList><TabPanel/></Tabs>` with Context) is only worth it if the interviewer asks for custom layouts. The keyboard math is the only piece pulled out, because it is pure and easy to test.

---

4. Design the state / data model

State or field	Purpose
`internalId`	Active tab id when the component is **uncontrolled**
`activeId` (prop)	Active tab id when the parent **controls** it
`tabRefs` (ref)	DOM nodes of the tab buttons, to move focus on arrow keys
`baseId` (`useId`)	Unique prefix for `id` / `aria-controls` / `aria-labelledby`

```
const baseId = useId();
const tabRefs = useRef<(HTMLButtonElement | null)[]>([]);
const [internalId, setInternalId] = useState(defaultActiveId);

const requestedId = activeId ?? internalId;
const selectedId = items.some((t) => t.id === requestedId && !t.disabled)
  ? requestedId
  : items.find((t) => !t.disabled)?.id;
const selectedIndex = items.findIndex((t) => t.id === selectedId);
```

Senior note:

- Store **only the id**. `selectedIndex` , "is this tab selected", the fallback tab and each `tabIndex` are all **derived** during render. Storing the index too would go stale when `items` is reordered.
- Store the **id, not the index**: if a tab is inserted before the active one, an index would silently point at a different tab.
- **Ownership:** if `activeId` is passed, the parent owns the value and `internalId` is ignored; otherwise `Tabs` owns it. Focus is owned by the DOM, we only move it.
- No `useEffect` to "fix" an invalid id: the fallback is computed every render, so there is no flash of a wrong tab.

---

5. Design the data structures and contracts

Example data

```
export const TABS: TabItem[] = [
  { id: 'profile', label: 'Profile', content: <p>Name, avatar and bio live here.</p> },
  { id: 'settings', label: 'Settings', content: <label>Display name <input /></label> },
  { id: 'billing', label: 'Billing', content: <p>Locked on the free plan.</p>, disabled: true },
```

```
  { id: 'notifications', label: 'Notifications', content: <p>Choose what we email you about.</p> },
];
```

Why stable string ids: they are the React `key`, the selection value, and part of the DOM ids. An array index would break all three when tabs are added, removed or reordered.

Component contract

```
type TabItem = {
  id: string;          // stable, unique
  label: string;       // text on the tab button
  content: ReactNode;  // panel body
  disabled?: boolean;
};

type TabsProps = {
  items: TabItem[];
  activeId?: string;         // controlled mode
  defaultActiveId?: string;  // uncontrolled starting tab
  onChange?: (id: string) => void;
  keepMounted?: boolean;     // keep hidden panels in the DOM
  label?: string;            // aria-label of the tablist
};
```

Usage:

```
// uncontrolled
<Tabs items={TABS} defaultActiveId="settings" />

// controlled
const [id, setId] = useState('profile');
<Tabs items={TABS} activeId={id} onChange={setId} />
```

There is no API call; if tabs came from a server, the fetch would live in the parent and pass `items` down.

---

6. Trace the data flow

Key interaction: user presses **ArrowRight** while "Settings" is focused (Billing is disabled).

```
keydown "ArrowRight" bubbles to div[role=tablist]
        |
        ▼
handleKeyDown → getNextIndex(items, selectedIndex = 1, +1)
        |        index 2 (Billing) disabled → skip → index 3
```

```
        ▼
select(3)
        │  id = "notifications" (≠ selectedId)
        │  uncontrolled → setInternalId("notifications")
        │  onChange?.("notifications")   (controlled parent updates activeId)
        ▼
tabRefs.current[3].focus()   (side effect: DOM focus moves)
        │
        ▼
re-render: selectedId → selectedIndex = 3
        │
        ▼
Notifications tab: aria-selected=true, tabIndex=0; others tabIndex=-1
Notifications panel visible; previous panel unmounted (or hidden if keepMounted)
```

---

7. Important technical challenges

Challenge 1: Keyboard navigation that skips disabled tabs and wraps

Problem: Tabs `[Profile, Settings, Billing(disabled), Notifications]`. ArrowRight from Settings must land on Notifications, ArrowRight from Notifications must wrap to Profile, and Home must skip a disabled first tab. Simple `index + 1` breaks all three cases, and negative `%` in JS returns a negative number.

Solution: One pure function that walks up to `n` steps in a direction, using a safe modulo, and returns the first enabled index. Home and End reuse it by starting just outside the list (`-1` going forward, `n` going backward).

```
export function getNextIndex(items: { disabled?: boolean }[], from: number, step: 1 | -1): number {
  const n = items.length;
  for (let i = 1; i <= n; i++) {
    const index = (((from + step * i) % n) + n) % n;
    if (!items[index].disabled) return index;
  }
  return -1; // every tab disabled
}

// Home: getNextIndex(items, -1, 1)   End: getNextIndex(items, items.length, -1)
```

Challenge 2: Accessibility (roles, ids, roving tabindex)

Problem: A screen reader user hears "button, button, button" and has no idea these are tabs or which panel belongs to which. A keyboard user has to press `Tab` through every tab button before reaching the content.

Solution: The WAI-ARIA tabs pattern. Roles tell assistive tech what it is; `aria-selected` says which is active; `aria-controls` / `aria-labelledby` link tab and panel; `useId` makes ids unique even with two `<Tabs>` on a

page. **Roving tabindex**: only the active tab has `tabIndex=0`, so the whole tablist is one tab stop and arrows move inside it.

```
<div role="tablist" aria-label={label} onKeyDown={handleKeyDown}>
  {items.map((tab, i) => (
    <button
      role="tab"
      id={`${baseId}-tab-${tab.id}`}
      aria-selected={i === selectedIndex}
      aria-controls={`${baseId}-panel-${tab.id}`}
      tabIndex={i === selectedIndex ? 0 : -1}
      disabled={tab.disabled}
      ...
```

Challenge 3: Controlled vs uncontrolled with a safe fallback

Problem: Some parents want to own the active tab (sync with URL, a wizard), others just want it to work. And a parent may pass `defaultActiveId="billing"` (disabled) or an id that no longer exists after `items` changed.

Solution: Like a native `<input value>` / `defaultValue`: if `activeId` is given, use it; otherwise use internal state. Always notify via `onChange`. Validate during render and fall back to the first enabled tab.

```
const requestedId = activeId ?? internalId;
const selectedId = items.some((t) => t.id === requestedId && !t.disabled)
  ? requestedId
  : items.find((t) => !t.disabled)?.id;

function select(index: number) {
  const id = items[index].id;
  if (id === selectedId) return;
  if (activeId === undefined) setInternalId(id);
  onChange?.(id);
}
```

Challenge 4: Keeping panel state when switching

Problem: The user types a name in Settings, peeks at Profile, comes back, and the input is empty, because the Settings panel unmounted.

Solution: `keepMounted` renders every panel and hides the inactive ones with the `hidden` attribute (also hidden from screen readers). Default stays lazy so heavy panels don't mount until opened.

```
{items.map((tab, i) => {
  const selected = i === selectedIndex;
  if (!selected && !keepMounted) return null;
```

```
  return <div role="tabpanel" hidden={!selected} ...>{tab.content}</div>;
})}
```

---

8. Implement in the correct order

#	Step	Working result
1	Define `TabItem` type and sample `TABS` data	Data ready
2	`Tabs` renders buttons from `items` and the first panel	Static tabs on screen
3	`useState` for active id, `onClick` to switch	Click switching works
4	Basic CSS: underline active tab, disabled style	Looks like tabs
5	Disabled tabs: `disabled` attribute + fallback to first enabled	Disabled can't be chosen
6	ARIA roles, `useId`-based ids, `aria-selected` / `aria-controls`	Screen-reader friendly
7	Roving `tabIndex`	One tab stop for the tablist
8	`getNextIndex` + `onKeyDown` for arrows, Home, End; move focus via refs	Full keyboard support
9	Controlled mode (`activeId` + `onChange`)	Parent can own the value
10	`keepMounted` with `hidden`	Panel state survives switches
11	Empty list message, focus-visible ring, overflow scroll	Edge cases covered

---

9. Cover all states and edge cases

State / case	Handling
Initial	First enabled tab, or `defaultActiveId` if valid
Switching (success)	New tab active, panel shown, `onChange` fired once
Empty `items`	"No tabs to show."
Single tab	Arrows keep focus on it (`getNextIndex` returns the same index)
Disabled tab	Not clickable, not focusable, skipped by arrows/Home/End
First or last tab disabled	Home/End land on the first/last **enabled** tab
All tabs disabled	No tab selected, no panel shown, `getNextIndex` returns `-1` so keys do nothing
`defaultActiveId` unknown or disabled	Fallback to first enabled tab
Active tab removed from `items`	Fallback recomputed on the next render, no stale index
Clicking the active tab	No state change, no extra `onChange`
Two `<Tabs>` on one page	`useId` keeps ids unique
Many tabs, narrow screen	Tab list scrolls horizontally
Panel with form input	Lost on switch by default; kept with `keepMounted`
Other keys (letters, Enter)	Ignored, no `preventDefault`

10. Final structure

```
_29_tab_components/
└── src/
    ├── components/
    │   └── Tabs/
    │       ├── Tabs.tsx        // component + TabItem type
    │       └── Tabs.css
    ├── utils/
    │   └── getNextIndex.ts     // pure keyboard navigation logic
    ├── data/
    │   └── tabs.tsx            // sample tab items
    ├── App.tsx                 // demo (controlled usage)
    ├── App.css
    ├── index.css
    └── main.tsx                // unchanged Vite entry
```

11. Full working code

`src/utils/getNextIndex.ts`

```
// Next enabled index from `from`, moving by `step` and wrapping around; -1 if every tab is disabled.
export function getNextIndex(items: { disabled?: boolean }[], from: number, step: 1 | -1): number {
  const n = items.length;
  for (let i = 1; i <= n; i++) {
    const index = (((from + step * i) % n) + n) % n;
    if (!items[index].disabled) return index;
  }
  return -1;
}
```

`src/components/Tabs/Tabs.tsx`

```
import { useId, useRef, useState, type KeyboardEvent, type ReactNode } from 'react';
import { getNextIndex } from '../../utils/getNextIndex';
import './Tabs.css';

export type TabItem = {
  id: string;
  label: string;
  content: ReactNode;
  disabled?: boolean;
};

type TabsProps = {
  items: TabItem[];
  activeId?: string; // controlled mode
  defaultActiveId?: string; // uncontrolled starting tab
  onChange?: (id: string) => void;
  keepMounted?: boolean; // keep hidden panels in the DOM so their state survives a switch
  label?: string; // accessible name of the tablist
};

export function Tabs({
  items,
  activeId,
  defaultActiveId,
  onChange,
  keepMounted = false,
  label = 'Tabs',
}: TabsProps) {
  const baseId = useId();
```

```
  const tabRefs = useRef<(HTMLButtonElement | null)[]>([]);
  const [internalId, setInternalId] = useState(defaultActiveId);

  const requestedId = activeId ?? internalId;
  // derived, not stored: fall back to the first enabled tab if the requested one is missing or
disabled
  const selectedId = items.some((t) => t.id === requestedId && !t.disabled)
    ? requestedId
    : items.find((t) => !t.disabled)?.id;
  const selectedIndex = items.findIndex((t) => t.id === selectedId);

  if (items.length === 0) return <p className="tabs-empty">No tabs to show.</p>;

  function select(index: number) {
    const id = items[index].id;
    if (id === selectedId) return;
    if (activeId === undefined) setInternalId(id);
    onChange?.(id);
  }

  function handleKeyDown(e: KeyboardEvent<HTMLDivElement>) {
    const nextIndex =
      e.key === 'ArrowRight' ? getNextIndex(items, selectedIndex, 1)
      : e.key === 'ArrowLeft' ? getNextIndex(items, selectedIndex, -1)
      : e.key === 'Home' ? getNextIndex(items, -1, 1)
      : e.key === 'End' ? getNextIndex(items, items.length, -1)
      : -1;
    if (nextIndex === -1) return;
    e.preventDefault();
    select(nextIndex);
    tabRefs.current[nextIndex]?.focus();
  }

  return (
    <div className="tabs">
      <div role="tablist" aria-label={label} className="tabs-list" onKeyDown={handleKeyDown}>
        {items.map((tab, i) => (
          <button
            key={tab.id}
            ref={(el) => {
              tabRefs.current[i] = el;
            }}
            id={`${baseId}-tab-${tab.id}`}
            type="button"
            role="tab"
            aria-selected={i === selectedIndex}
            aria-controls={`${baseId}-panel-${tab.id}`}
```

```
            tabIndex={i === selectedIndex ? 0 : -1}
            disabled={tab.disabled}
            className="tabs-tab"
            onClick={() => select(i)}
          >
            {tab.label}
          </button>
        ))}
      </div>

      {items.map((tab, i) => {
        const selected = i === selectedIndex;
        if (!selected && !keepMounted) return null;
        return (
          <div
            key={tab.id}
            id={`${baseId}-panel-${tab.id}`}
            role="tabpanel"
            aria-labelledby={`${baseId}-tab-${tab.id}`}
            hidden={!selected}
            tabIndex={0}
            className="tabs-panel"
          >
            {tab.content}
          </div>
        );
      })}
    </div>
  );
}
```

src/components/Tabs/Tabs.css

```
.tabs-list {
  display: flex;
  gap: 4px;
  border-bottom: 1px solid #d0d4dc;
  overflow-x: auto;
}

.tabs-tab {
  padding: 10px 16px;
  border: 0;
  border-bottom: 2px solid transparent;
  background: none;
  font: inherit;
```

```
  color: #555;
  white-space: nowrap;
  cursor: pointer;
}

.tabs-tab[aria-selected='true'] {
  color: #2b59c3;
  border-bottom-color: #2b59c3;
  font-weight: 600;
}

.tabs-tab:hover:not(:disabled) {
  color: #1a1a1a;
}

.tabs-tab:disabled {
  color: #aaa;
  cursor: not-allowed;
}

.tabs-tab:focus-visible,
.tabs-panel:focus-visible {
  outline: 2px solid #2b59c3;
  outline-offset: -2px;
}

.tabs-panel {
  padding: 16px 4px;
}

.tabs-empty {
  color: #777;
}
```

src/data/tabs.tsx

```
import type { TabItem } from '../components/Tabs/Tabs';

export const TABS: TabItem[] = [
  { id: 'profile', label: 'Profile', content: <p>Name, avatar and bio live here.</p> },
  {
    id: 'settings',
    label: 'Settings',
    content: (
      <label>
        Display name <input placeholder="Type, then switch tabs" />
```

```
      </label>
    ),
  },
  { id: 'billing', label: 'Billing', content: <p>Billing is locked on the free plan.</p>, disabled:
true },
  { id: 'notifications', label: 'Notifications', content: <p>Choose what we email you about.</p> },
];
```

src/App.tsx

```
import { useState } from 'react';
import { Tabs } from './components/Tabs/Tabs';
import { TABS } from './data/tabs';
import './App.css';

export default function App() {
  const [activeId, setActiveId] = useState('profile');

  return (
    <main className="app">
      <h1>Tab Component</h1>
      <Tabs items={TABS} activeId={activeId} onChange={setActiveId} keepMounted label="Account
settings" />
      <p className="app-status">
        Active tab: <strong>{activeId}</strong>
      </p>
    </main>
  );
}
```

src/App.css

```
.app {
  max-width: 640px;
  margin: 48px auto;
  padding: 0 16px;
}

.app-status {
  color: #555;
}

.app input {
  margin-left: 8px;
  padding: 6px 8px;
```

```
  font: inherit;
}
```

`src/index.css`

```
:root {
  font-family: system-ui, Avenir, Helvetica, Arial, sans-serif;
  line-height: 1.5;
  color: #1a1a1a;
  background: #f6f7f9;
}

body {
  margin: 0;
}
```

`src/main.tsx` stays as generated by Vite.

---

12. Wrap-up

Edge cases handled

- Disabled tabs are skipped by click, Tab key, arrows, Home and End.
- Arrow keys wrap around; single-tab and all-disabled lists don't crash.
- Invalid / disabled / removed active id falls back to the first enabled tab during render.
- Empty list shows a message.
- Unique ids per instance via `useId`.
- Optional state preservation with `keepMounted`.

Trade-offs to mention

- **Automatic vs manual activation:** automatic is faster for users when panels are cheap; if a panel fetches data or is heavy, manual activation (arrows move focus, Enter/Space selects) avoids loading every tab you pass over.
- **Data-driven `items` vs compound components:** `items` is simpler and easy to validate; compound components (`<Tab>`, `<TabPanel>` + Context) give layout freedom at the cost of more code.
- **Lazy vs keepMounted:** lazy saves memory and work; keepMounted keeps state but every panel stays mounted (and runs its effects).
- **Native `disabled` vs `aria-disabled`:** native removes the tab from focus entirely (simple). `aria-disabled` keeps it focusable so screen-reader users can discover it but needs manual click guarding.
- With lazy panels, `aria-controls` on inactive tabs points to a panel that is not in the DOM; acceptable here, fixable by always rendering empty hidden panels.

What to add with more time

- Unit tests for `getNextIndex` and React Testing Library tests for click, keyboard and ARIA attributes.
- Manual activation mode, vertical orientation (Up/Down keys, `aria-orientation`).
- URL sync (`?tab=settings`) so refresh and share keep the tab.
- Animated active indicator, scroll the active tab into view on overflow.
- Closable / draggable tabs, lazy-loaded panel content with `Suspense`.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you pick automatic activation instead of manual?

The ARIA guide recommends automatic when panels show instantly: fewer keystrokes. If panels are expensive (network, heavy charts), I would switch to manual so arrowing past a tab doesn't trigger its work. It is a small change: arrows only move focus, Enter/Space call `select`.

Q2. What if the product wants the active tab to survive a page refresh?

Use controlled mode in the parent and read/write `?tab=` with `URLSearchParams` (or the router). `Tabs` itself doesn't change: it already takes `activeId` and reports `onChange`.

Q3. Why an `items` prop and not `<Tab>` / `<TabPanel>` children?

Faster to build, easy to validate, and keyboard logic gets a plain array. Compound components are better when consumers need custom markup between the list and panels; I would build them with a Context holding `selectedId` and `select`.

State design

Q4. Why store the id and not the index?

An index is a position, not an identity. If a tab is inserted before the active one, the index now points to another tab. The id stays correct; the index is derived each render with `findIndex`.

Q5. Why not fix an invalid `defaultActiveId` in a `useEffect`?

An effect runs after paint, so the user would see a wrong or empty panel for one frame, plus an extra render. Deriving the fallback during render is correct on the first paint and can never get out of sync.

Q6. What happens if the parent passes `activeId` but no `onChange`?

It behaves like a read-only controlled input: clicks call `onChange` (undefined) and the selection never changes. That's the expected React contract; in a library I'd add a dev warning.

Q7. What if a component switches between controlled and uncontrolled?

It works here but it's a smell: `internalId` would hold an old value. React warns about this for inputs; a library would warn too. Pick one mode per instance.

Keyboard challenge

Q8. Why `((x % n) + n) % n` instead of `x % n`?

In JS, `-1 % 4` is `-1`, not `3`. Adding `n` and taking `%` again turns negatives into the correct wrapped index.

Q9. Why is `onKeyDown` on the tablist and not on each tab?

Event delegation: one handler, keys from any tab bubble up to it. It also keeps the per-tab JSX small.

Q10. Why call `preventDefault` only for handled keys?

Arrow keys and Home/End scroll the page by default, so we stop that. Other keys (Tab, letters, Ctrl+C) must keep their browser behaviour, so we return early without blocking them.

Q11. Why move focus with refs? Couldn't state alone do it?

Focus is DOM state, not React state. Changing `tabIndex` only decides *which* element is focusable; `element.focus()` actually moves the keyboard cursor. Refs are the right tool for that one side effect.

Accessibility challenge

Q12. What is roving tabindex and why use it?

Only the active tab has `tabIndex=0`, all others `-1`. The tablist becomes one stop in the Tab order and arrows move within it, matching how native radio groups and OS tab controls behave.

Q13. Why `useId` instead of `tab-${tab.id}`?

Two `<Tabs>` on one page with the same item ids would produce duplicate DOM ids and break `aria-controls`. `useId` gives a unique, SSR-safe prefix per instance.

Q14. Why give the panel `tabIndex={0}`?

If a panel has no focusable content (just text), keyboard users could not reach it after the tablist; `Tab` would skip to the next page element. The ARIA pattern recommends it.

Implementation and performance

Q15. What if there are 50 tabs?

Rendering 50 buttons is fine. The UX is the problem: the list scrolls horizontally (`overflow-x: auto`); I'd add "scroll active tab into view" and maybe an overflow "More" menu. Panels stay lazy so only one renders.

Q16. What if a panel's content is slow to load?

Keep it lazy, load inside the panel with its own loading/error state, or `React.lazy` + `Suspense` per panel. Consider manual activation so arrowing past it doesn't start the load.

Q17. Doesn't the inline `ref` callback and `onClick` arrow re-create functions each render?

Yes, and it doesn't matter for a handful of buttons: no measurable cost. `useCallback` + `memo` would add complexity for no gain here.

Q18. Why `hidden` and not `display: none` via a class for keepMounted?

`hidden` is semantic: it hides from screen readers and needs no CSS. Same visual result, less code.

Trade-offs and "why not X"

Q19. Why not use a library like Radix or Headless UI?

In production I would, they handle many edge cases. In an interview the point is to show I understand the pattern (ARIA, roving focus, controlled state), and these rounds usually expect it built from scratch.

Q20. Why native `disabled` and not `aria-disabled`?

Native is simpler: no click guard needed and it's skipped by Tab. The downside is screen-reader users can't land on it to learn it exists. If discoverability matters I'd use `aria-disabled`, guard `select`, and still skip it in arrow navigation.

Q21. How would you test this?

Unit-test `getNextIndex` (wrap, skip disabled, all disabled, Home/End). With React Testing Library: click switches panels, `userEvent.keyboard('{ArrowRight}')` moves focus and `aria-selected`, disabled tab ignored, `onChange` called once, typed input survives with `keepMounted`.